LeetCode Practice

13 Easy: Stack

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

13
EASY

Patterns (in order)

1. Stack (13)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

13 questions | tap a title to jump to it

Easy questions (13) Go >

Stack (13) Go >

E	1. Valid Parentheses	LC 20
E	2. Binary Tree Inorder Traversal	LC 94
E	3. Binary Tree Preorder Traversal	LC 144
E	4. Binary Tree Postorder Traversal	LC 145
E	5. Implement Stack Using Queues	LC 225
E	6. Implement Queue using Stacks	LC 232
E	7. Next Greater Element I	LC 496
E	8. Baseball Game	LC 682
E	9. Backspace String Compare	LC 844
E	10. Number of Recent Calls	LC 933
E	11. Remove All Adjacent Duplicates In String	LC 1047
E	12. Final Prices With a Special Discount in a Shop	LC 1475
E	13. Maximum Nesting Depth of the Parentheses	LC 1614

Easy questions (13)

EASY | TOPIC 1 OF 1

Stack

13 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Valid Parentheses LC 20
2. Binary Tree Inorder Traversal LC 94
3. Binary Tree Preorder Traversal LC 144
4. Binary Tree Postorder Traversal LC 145
5. Implement Stack Using Queues LC 225
6. Implement Queue using Stacks LC 232
7. Next Greater Element I LC 496
8. Baseball Game LC 682
9. Backspace String Compare LC 844
10. Number of Recent Calls LC 933
11. Remove All Adjacent Duplicates In String LC 1047
12. Final Prices With a Special Discount in a Shop LC 1475
13. Maximum Nesting Depth of the Parentheses LC 1614

EASY | Stack | LeetCode #20 | 1 of 13 | 1/13

Valid Parentheses

QUESTION

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

CONSTRAINTS

- 1. Open brackets must be closed by the same type of brackets.
- 2. Open brackets must be closed in the correct order.
- 3. Every close bracket has a corresponding open bracket of the same type.

Example 1	
Input	`s = "()"`
Output	`true`

Example 2	
Input	`s = "()[]{}"`
Output	`true`

Example 3	
Input	`s = "(]"`
Output	`false`

Example 4	
Input	`s = "([])"`
Output	`true`

Example 5	
Input	`s = "([)]"`
Output	`false`

- 1 <= s.length <= 10^4
- s consists of parentheses only '()[]{}'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

A string of brackets is valid if: every opening bracket is closed by the same type,

every closing bracket has a corresponding opener, and the order is correct.

Return true if valid, false otherwise. Right?"

"A few quick questions:

Can the string be empty? Yes — empty string is valid.

Only bracket characters — no spaces or letters? Yes, the constraints confirm this."

PHASE 1 CLARIFY

"Let me walk the examples:

'()[]{}' -> each pair closes correctly -> true ok

'([)]' -> '[' is open when ')' arrives -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while the string contains '()', '[]' or '{}':
    delete one occurrence
return the string is empty
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every deletion rescans and copies the string, and there can be n/2 of them. A stack that pushes openers and pops on a matching closer is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is repeatedly scanning the string.

A stack solves this in a single pass:

push every opening bracket; on a closing bracket, check that the top matches.

If it doesn't — or the stack is empty — it's invalid.

At the end the stack must be empty (every opener was closed).

Pseudocode:

```
stack = []
for ch in s:
if ch in '([{': stack.append(ch)
else:
if not stack or stack[-1] != matching_opener: return
  False
stack.pop()
return stack == []
```

Time: O(n). Space: O(n) for the stack.

Does that make sense before I code it?"

Key insight: the stack represents the sequence of expected closing brackets at the current nesting depth — each character is pushed or triggers a match check exactly once.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _20_ValidParentheses:
    def isValid(self, s):
```

```
        pairs = {')': '(', ']': '[', '}': '{'}
        stack = []
        for ch in s:
            if ch not in pairs:
                stack.append(ch)
            elif not stack or stack[-1] != pairs[ch]:
                return False
            else:
                stack.pop()
        return not stack
```

PHASE 5 TEST & DEBUG

"Let me trace isValid('()[]{}')."

stack=[]. Push '('->['(']. Push ')'->top='(' match->pop. stack=[].

Push '['->['[']. Push ']'->top='['->pop. stack=[].

Push '{'->['{']. Push '}'->top='{'->pop. stack=[].

Empty stack -> return True ok

'([)]': push '('->['(']. '['->['(','[']. ')'->top='['!='(' pair -> return False ok

WHY stack: each opening bracket expects a matching close in LIFO order.

Edge: single char '(' -> stack=['(']. Never popped -> return False ok

WHY stack (LIFO): closing bracket must match most recently opened -> stack gives most recent open.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass	worst case all openers e.g. '((((('

Brute O(n²) is easy to understand but TLEs for long strings.

Follow-up: Minimum Add to Make Parentheses Valid (#921) — count unmatched openers/closers.

Follow-up: Longest Valid Parentheses (#32) — find the longest valid substring using DP or stack.

EASY | Stack | LeetCode #94 | 2 of 13 | 2/13

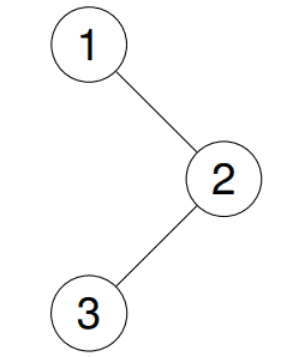
Binary Tree Inorder Traversal

QUESTION

Given the root of a binary tree, return the inorder traversal of its nodes' values.

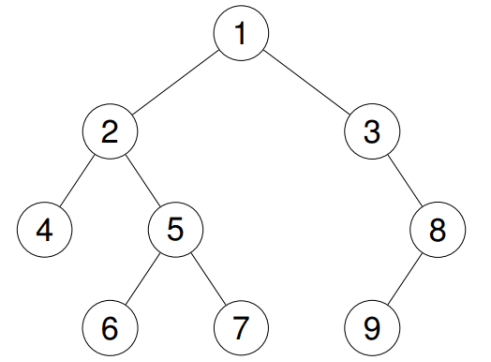
Example 1

Input	`root = [1,null,2,3]`
Output	`[1,3,2]`



Example 2

Input	`root = [1,2,3,4,5,null,8,null,null,6,7,9]`
Output	`[4,2,6,5,7,1,3,9,8]`



Example 3

Input	`root = []`
Output	`[]`

Example 4

Input	`root = [1]`
Output	`[1]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100
- Follow up: Recursive solution is trivial, could you do it iteratively?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the inorder traversal of a binary tree: left subtree, root, right subtree. Right?"

"A few quick questions:

Follow-up asks for iterative solution? Yes — avoids recursion stack.

Empty tree -> []?"

"Let me walk: root=[1,null,2,3] -> [1,3,2] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def visit(node):
    if empty return
    visit(left)
    append node.val
    visit(right)
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: every node is visited once. The cost is the recursion stack (O(n) for a skewed tree); Morris traversal does it in O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the implicit recursion stack.

Iterative with explicit stack: push all left nodes, then pop and process,

then push right subtree's left nodes.

Time: O(n). Space: O(h)."

Key insight: the explicit stack simulates the implicit recursion stack — pushing all left-spine nodes first ensures inorder processing without recursion overhead.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _94_BinaryTreeInorder:
    def inorderTraversal(self, root):
        result = []
        stack = []
        current = root
        while current or stack:
            while current:
```

```
            stack.append(current)
            current = current.left
        current = stack.pop()
        result.append(current.val)
        current = current.right
    return result
```

PHASE 5 TEST & DEBUG

"Let me trace inorder traversal on [1,null,2,3] (root=1, right=2, right.left=3)."

`inorder(1): recurse left -> inorder(None) -> return.`

Visit 1 -> result=[1].

recurse right -> inorder(2): recurse left -> inorder(3): recurse left->None; visit 3->result=[1,3]; recurse right->None.

Back at 2: visit 2->result=[1,3,2]; recurse right->None.

Final: [1,3,2] ok (left subtree, root, right subtree)

Edge: empty tree -> inorder(None) returns immediately -> [] ok

Edge: single node [5] -> inorder(5): no left, visit 5, no right -> [5] ok

WHY this order: BST inorder = sorted ascending; general BT inorder = L-N-R traversal.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** stack holds at most h nodes

Morris traversal: O(n) time, O(1) space — modifies tree temporarily.

Follow-up: Validate BST (#98) — inorder of BST must be strictly increasing.

Follow-up: Kth Smallest in BST (#230) — stop inorder at kth element.

EASY | Stack | LeetCode #144 | 3 of 13 | 3/13

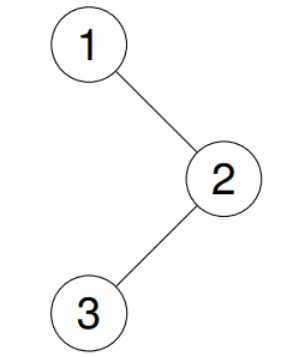
Binary Tree Preorder Traversal

QUESTION

Given the root of a binary tree, return the preorder traversal of its nodes' values.

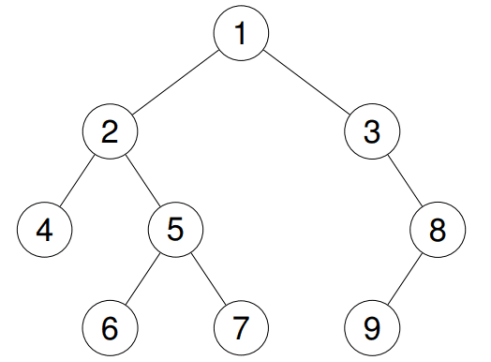
Example 1

Input	`root = [1,null,2,3]`
Output	`[1,2,3]`



Example 2

Input	`root = [1,2,3,4,5,null,8,null,null,6,7,9]`
Output	`[1,2,4,5,6,7,3,8,9]`



Example 3

Input	`root = []`
Output	`[]`

Example 4

Input	`root = [1]`
Output	`[1]`

CONSTRAINTS

- The number of nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100
- Follow up: Recursive solution is trivial, could you do it iteratively?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the preorder traversal of a binary tree: root, then left subtree, then right subtree. Right?"

"A few quick questions:

Iterative or recursive? Both are valid — follow-up asks for iterative.

Empty tree -> []? Yes."

"Let me walk: root=[1,null,2,3] -> [1,2,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def visit(node):
    if empty return
    append node.val
    visit(left)
    visit(right)
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: each node is visited once. The cost is the recursion stack (O(n) when skewed); an explicit stack is the same, and Morris traversal uses O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(h) recursion stack — iterative avoids stack overflow risk.

Use an explicit stack: push root, then for each popped node push right then left

(so left is processed first).

Time: O(n). Space: O(h) explicit stack.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _144_BinaryTreePreorder:
    def preorderTraversal(self, root):
        if not root:
            return []
        result = []
        stack = [root]
        while stack:
            node = stack.pop()
            result.append(node.val)
            if node.right: stack.append(node.right)
```

```
            if node.left: stack.append(node.left)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace preorderTraversal on [1,null,2,3]."

Tree: 1's left=null, right=2; 2's left=3, right=null

Iterative with stack: stack=[1], result=[]

Pop 1: result=[1]. Push right(2) then left(null skipped). stack=[2]

Pop 2: result=[1,2]. Push right(null skipped), left(3). stack=[3]

Pop 3: result=[1,2,3]. No children. stack=[]

Return [1,2,3] ok

WHY push right before left: stack is LIFO, so left gets processed first (preorder: root-left-right).

Recursive equiv: visit node, recurse left, recurse right.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)** stack depth = tree height

Recursive is simpler; iterative avoids Python's recursion limit for deep trees.

Follow-up: Inorder (#94) and Postorder (#145) — similar stack-based patterns.

Follow-up: Morris traversal achieves O(1) space using thread pointers.

EASY | Stack | LeetCode #145 | 4 of 13 | 4/13

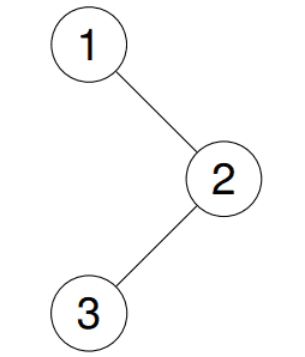
Binary Tree Postorder Traversal

QUESTION

Given the root of a binary tree, return the postorder traversal of its nodes' values.

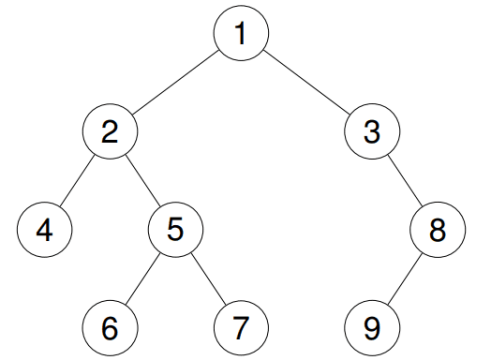
Example 1

Input	`root = [1,null,2,3]`
Output	`[3,2,1]`



Example 2

Input	`root = [1,2,3,4,5,null,8,null,null,6,7,9]`
Output	`[4,6,7,5,2,9,8,3,1]`



Example 3

Input	`root = []`
Output	`[]`

Example 4

Input	`root = [1]`
Output	`[1]`

CONSTRAINTS

- The number of the nodes in the tree is in the range [0, 100].
- 100 <= Node.val <= 100
- Follow up: Recursive solution is trivial, could you do it iteratively?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the postorder traversal: left subtree, right subtree, then root. Right?"

"A few quick questions:

Iterative follow-up? Yes. Empty tree -> []?"

"Let me walk: root=[1,null,2,3] -> [3,2,1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def visit(node):
    if empty return
    visit(left)
    visit(right)
    append node.val
```

TIME	SPACE
O(n)	O(h)

NOTE

Not slow: each node is visited once. The cost is the recursion stack (O(n) when skewed); reversing a modified pre-order or Morris traversal uses less stack.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the recursion stack.

Iterative trick: preorder visits root->left->right. Postorder = reverse of root->right->left.

Use a preorder-like traversal pushing right then left, collect, reverse at end.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _145_BinaryTreePostorder:
    def postorderTraversal(self, root):
        if not root:
            return []
        result = []
        stack = [root]
        while stack:
            node = stack.pop()
            result.append(node.val)
            if node.left: stack.append(node.left)
            if node.right: stack.append(node.right)
        return result[::-1]
```

PHASE 5 TEST & DEBUG

"Let me trace postorderTraversal on [1,null,2,3]."

Tree: root=1, right=2, 2.left=3

Iterative trick: collect reversed preorder (root-right-left), then reverse

stack=[1], result=[]

Pop 1: result=[1]. Push left(null skip), right(2). stack=[2]

Pop 2: result=[1,2]. Push left(3), right(null skip). stack=[3]

Pop 3: result=[1,2,3]. stack=[]

Reverse result: [3,2,1] ok (postorder: left-right-root)

WHY reverse trick: avoids complex two-pass logic; mirrors preorder with left/right swapped.

Recursive: recurse left, recurse right, visit root.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for result + stack

The reverse trick is elegant — one line change from preorder.

Follow-up: for true O(h) space iterative, track the previously processed node.

Follow-up: Tree serialization often uses postorder for bottom-up processing.

EASY | Stack | LeetCode #225 | 5 of 13 | 5/13

Implement Stack Using Queues

QUESTION

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

CONSTRAINTS

- void push(int x) Pushes element x to the top of the stack.
- int pop() Removes the element on the top of the stack and returns it.
- int top() Returns the element on the top of the stack.
- boolean empty() Returns true if the stack is empty, false otherwise.

Notes:

- You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.
- 1 <= x <= 9
- At most 100 calls will be made to push, pop, top, and empty.
- All the calls to pop and top are valid.
- Follow-up: Can you implement the stack using only one queue?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement a stack (LIFO) using only two queues.

Support push, pop, top, and empty. Right?"

"A few quick questions:

All calls to pop and top are valid (stack non-empty)? Yes."

"Let me walk: push(1),push(2),top()->2,pop()->2,empty()->false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result.append(one_queue)
def push(x):
    enqueue
def pop():
    move n - 1 items to a second queue, remove
        the last one, swap the queues
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n) per pop	O(n)

NOTE

Not slow in the usual sense: with queues only, one of push or pop has to cost O(n). Rotating the single queue after each push makes pop and top O(1) and push O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Queue-based stack with O(1) push using rotation — O(n) push, O(1) pop. Walk-through:"

push(x): enqueue x. Then rotate the queue (n-1) times — dequeue front, enqueue to back.

After rotation: x is now at front of queue -> it will be the first dequeued = top of stack.

pop(): dequeue from front.

top(): peek front.

WHY rotate on push: queue is FIFO but stack is LIFO. By rotating new element to front,

the queue's front always holds the most recently pushed element.

Alternative: two-queue swap on pop (O(n) pop, O(1) push).

Time: O(n) push, O(1) pop. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _225_ImplementStackUsingQueues:
    def __init__(self):
        self._queue = deque()

    def push(self, x):
        self._queue.append(x)
        for _ in range(len(self._queue) - 1):
            self._queue.append(self._queue.popleft())

    def pop(self):
        return self._queue.popleft()

    def top(self):
        return self._queue[0]

    def empty(self):
        return not self._queue
```

PHASE 5 TEST & DEBUG

"Let me trace MyStack push(1),push(2),pop()."

Use one queue; on each push, rotate all previous elements to back

push(1): q=[1]

push(2): enqueue 2 -> q=[1,2]; rotate 1 element to back: dequeue 1, enqueue 1 -> q=[2,1]

PHASE 5 TEST & DEBUG

```
top() = q[0] = 2 ok
pop(): dequeue -> return 2, q=[1] ok
push(3): q=[1,3] -> rotate 1: dequeue 1, enqueue 1 ->
  q=[3,1]
```

WHY rotate on push: makes front of queue always the most recently pushed element.

Edge: single push -> no rotation needed; pop/top work directly.

stack=[3,2,1] (top=3). push(3): enqueue 3 to q=[1,2,3], then rotate: move '1','2' to back -> q=[3,1,2]. stack.pop()=dequeue front=3 ok. stack=[2,1] after pop.

PHASE 6 COMPLEXITY & FOLLOW-UP

push O(n), pop/top O(1). Space O(n).

Alternative: make pop O(n) and push O(1) — move all but last on pop.

Follow-up: Implement Queue using Stacks (#232) — symmetric design.

Follow-up: if only one queue allowed, this rotation trick is the standard answer.

IN YOUR APP | Expose | Editorial | Community

EASY | Stack | LeetCode #232 | 6 of 13 | 6/13

Implement Queue using Stacks

QUESTION

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

CONSTRAINTS

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.
- 1 <= x <= 9
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.
- Follow-up: Can you implement the queue such that each operation is amortized O(1) time complexity? In other words, performing n operations will take overall O(n) time even if one of those operations may take longer.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement a FIFO queue using only stack operations (push/pop/peek from the top).

Support push(x), pop(), peek(), and empty().

The follow-up asks for amortized O(1) per operation. Right?"

"A few quick questions:

Is it guaranteed that pop and peek are only called on non-empty queues? Yes.

Standard stack operations only — no deque or collections.deque tricks?"

"Let me walk: push(1), push(2), peek()->1, pop()->1, empty()->false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep two stacks, input and output
def pop():
    move every item from input to output, pop
        from output, move everything back
```

TIME	SPACE
O(n) per pop	O(n)

WHY IT IS SLOW

Moves all elements on every pop. Move items to the output stack only when it is empty and leave them there: each element moves at most twice, O(1) amortised.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is moving all elements on every push — O(n) per push.

Two stacks — inbox and outbox — give amortized O(1):

push always goes to inbox.

For pop/peek: if outbox is empty, pour all of inbox into outbox (reverses order).

Then pop/peek from outbox.

Each element moves at most twice total, so the amortized cost per operation is O(1).

Time: O(1) amortized all ops. Space: O(n) across both stacks.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _232_QueueUsingStacks:
    def __init__(self):
        self._inbox = []
        self._outbox = []

    def push(self, x):
        self._inbox.append(x)

    def _shift(self):
        if not self._outbox:
            while self._inbox:
                self._outbox.append(self._inbox.pop())

    def pop(self):
        self._shift()
        return self._outbox.pop()

    def peek(self):
        self._shift()
        return self._outbox[-1]

    def empty(self):
        return not self._inbox and not self._outbox
```

PHASE 5 TEST & DEBUG

"Let me trace MyQueue using stacks: push(1),push(2),peek(),pop(),empty()."

in_stack for push, out_stack for pop. Transfer lazily.

```
push(1): in=[1]. push(2): in=[1,2].
peek(): out empty -> transfer: pop in->out=[2,1]. Peek
  out.top()=1 ok
pop(): out=[2,1]. Pop top=1. out=[2]. Return 1 ok
empty(): in=[], out=[2] -> not empty -> False ok
```

WHY lazy transfer: amortized O(1). Each element moves from in->out exactly once.

Transfer only when out is empty — never mid-operation.

Two stacks simulate queue: in_stack=[3], out_stack=[]. queue state: front=out_stack.peek()=1. When out empty: transfer in->out: out=[2,1], in=[3]. deque=[1,2,3] conceptually (FIFO) ok

PHASE 6 COMPLEXITY & FOLLOW-UP

All ops amortized O(1): each element transferred at most once across the two stacks.

Space O(n): elements split across inbox and outbox at any moment.

Follow-up: Implement Stack using Queues (#225) — symmetric design with one queue.

Follow-up: Design Circular Queue (#622) — fixed capacity, O(1) all ops with a ring buffer.

EASY | Stack | LeetCode #496 | 7 of 13 | 7/13

Next Greater Element I

QUESTION

The next greater element of some element x in an array is the first greater element that is to the right of x in the same array.

You are given two distinct 0-indexed integer arrays nums1 and nums2, where nums1 is a subset of nums2.

For each 0 <= i < nums1.length, find the index j such that nums1[i] == nums2[j] and determine the next greater element of nums2[j] in nums2. If there is no next greater element, then the answer for this query is -1.

Return an array ans of length nums1.length such that ans[i] is the next greater element as described above.

Example 1

Input	`nums1 = [4,1,2], nums2 = [1,3,4,2]`
Output	`[-1,3,-1]`
Explanation	*The next greater element for each value of nums1 is as follows: - 4 is underlined in nums2 = [1,3, 4 ,2]. There is no next greater element, so the answer is -1. - 1 is underlined in nums2 = [1 ,3,4,2]. The next greater element is 3. - 2 is underlined in nums2 = [1,3,4, 2]. There is no next greater element, so the answer is -1.*

Example 2

Input	`nums1 = [2,4], nums2 = [1,2,3,4]`
Output	`[3,-1]`
Explanation	*The next greater element for each value of nums1 is as follows: - 2 is underlined in nums2 = [1, 2 ,3,4]. The next greater element is 3. - 4 is underlined in nums2 = [1,2,3, 4]. There is no next greater element, so the answer is -1.*

CONSTRAINTS

- 1 <= nums1.length <= nums2.length <= 1000
- 0 <= nums1[i], nums2[i] <= 10^4
- All integers in nums1 and nums2 are unique.
- All the integers of nums1 also appear in nums2.
- Follow up: Could you find an O(nums1.length + nums2.length) solution?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each nums1[i], find the next greater element in nums2 (first element to its right that is larger).

Return -1 if none exists. Right?"

"A few quick questions:

nums1 is a subset of nums2? Yes. Order of nums1 determines output order? Yes."

"Let me walk: nums1=[4,1,2], nums2=[1,3,4,2] -> 4->-1, 1->3, 2->-1 -> [-1,3,-1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x in nums1:
    find x in nums2 and scan to its right for the
        first larger value
```

TIME	SPACE
O(m * n)	O(1)

WHY IT IS SLOW

Rescans nums2 for each query. One pass over nums2 with a monotonic stack records every value's next greater element in a hash map: O(m + n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack + hashmap precompute — O(n+m). Walk-through:"

Precompute next_greater for ALL elements of nums2:

Stack (decreasing). For each num in nums2:

While stack non-empty and stack[-1] < num:
next_greater[stack.pop()] = num.

`stack.push(num).`

Remaining in stack have no next greater -> next_greater[x] = -1.

For each x in nums1: answer.append(next_greater.get(x, -1)).

WHY precompute on nums2: nums1 is a subset of nums2; precomputing once covers all queries.

WHY monotone-decreasing stack: when a larger number appears, it's the 'next greater'

`for everything currently in the stack that is smaller.`

Time: O(n+m). Space: O(n) hashmap.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _496_NextGreaterElementI:
    def nextGreaterElement(self, nums1, nums2):
        next_greater = {}
        stack = []
        for num in nums2:
            while stack and stack[-1] < num:
                next_greater[stack.pop()] = num
            stack.append(num)
        for num in stack:
            next_greater[num] = -1
        return [next_greater[n] for n in nums1]
```

PHASE 5 TEST & DEBUG

"Let me trace nextGreaterElement([4,1,2],[1,3,4,2])."

Process nums2 with monotonic stack (decreasing). Build map: val->next_greater.

nums2=[1,3,4,2]. stack=[], map={}.

1: push -> [1]. 3: 3>1 -> pop 1, map[1]=3. push 3 -> [3].

4: 4>3 -> pop 3, map[3]=4. push 4 -> [4].

2: 2<4 -> push -> [4,2]. End: remaining [4,2] -> map[4]=-1, map[2]=-1.

map={1:3,3:4,4:-1,2:-1}.

Answer for nums1=[4,1,2]: [map[4],map[1],map[2]]=[-1,3,-1] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n+m)	**O(m)** for map

Follow-up: Next Greater Element II (#503) — circular array; iterate twice.

Follow-up: Daily Temperatures (#739) — return days to wait, not actual element.

IN YOUR APP | Expose | Editorial | Community

EASY | Stack | LeetCode #682 | 8 of 13 | 8/13

Baseball Game

QUESTION

You are keeping the scores for a baseball game with strange rules. At the beginning of the game, you start with an empty record.

You are given a list of strings operations, where operations[i] is the ith operation you must apply to the record and is one of the following:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Simulate a baseball game with integer score entries, '+' (sum of last two), 'D' (double last),

and 'C' (invalidate last). Return the total sum after all operations. Right?"

"A few quick questions:

Is the input always valid (e.g. '+' always has two previous scores)? Yes."

"Let me walk: ops=['5','2','C','D','+'] -> [5], [5,2], [5](C), [5,10](D), [5,10,15](+) -> 30 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
scores = a list
for operation in operations:
    apply it to the list
    after every operation re-sum the whole list
        at the end of the loop
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Re-summing is O(n) per operation. Keep a stack of valid scores and a running total updated at each push and pop: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Simulate with stack — O(n). Walk-through:"

For each op:

Integer: push it directly.

'+': pop top two, push their sum (don't remove originals — just peek? No, pop both, push sum).

Actually: pop top two, push sum, then push those two back? No —

PHASE 3 OPTIMIZE

'+': pop a=stack.pop(), b=stack.pop(); push(b+a); push(b); push(a)? No —

Actually: sum of last two scores: val = stack[-1] + stack[-2]; stack.append(val).

'D': double the top: val = stack[-1]*2; stack.append(val).

'C': invalidate last: stack.pop().

Return sum(stack).

WHY stack: each operation refers to 'last valid' scores — stack top = most recent score.

Invalidated scores are popped; new scores are pushed.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _682_BaseballGame:
    def calPoints(self, operations):
        record = []
        for op in operations:
            if op == '+':
                record.append(record[-1] + record[-2])
            elif op == 'D':
                record.append(record[-1] * 2)
            elif op == 'C':
                record.pop()
            else:
                record.append(int(op))
        return sum(record)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

ops=['5','2','C','D','+']:

'5'->[5]. '2'->[5,2]. 'C'->[5]. 'D'->[5,10]. '+'->[5,10,15]. sum=30 ok

ops=['5','-2','4','C','D','9','+','+']:

[5],[-2+5=-2],[4],[C->[-2,5]? wait:
[5,-2,4]->C->[5,-2]->D->[5,-2,-4]->9->[...] Hmm actually:

[5],[5,-2],[5,-2,4],C->[5,-2],D->[5,-2,-4],9->[5,-2,-4,9],+->[5,-2,-4,9,5],+->[5,-2,-4,9,5,14]=27 ok

Stack naturally maintains valid records for each operation type

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) single pass	O(n) at most n entries in record

Follow-up: if 'C' could invalidate multiple records, extend with a count parameter.

Follow-up: Valid Parentheses (#20) — same stack pattern for matching bracket checks.

IN YOUR APP | Expose | Editorial | Community

EASY | Stack | LeetCode #844 | 9 of 13 | 9/13

Backspace String Compare

QUESTION

Given two strings s and t, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1	
Input	`s = "ab#c", t = "ad#c"`
Output	`true`
Explanation	*Both s and t become "ac".*

Example 2	
Input	`s = "ab##", t = "c#d#"`
Output	`true`
Explanation	*Both s and t become "".*

Example 3	
Input	`s = "a#c", t = "b"`
Output	`false`
Explanation	*s becomes "c" while t becomes "b".*

CONSTRAINTS

- 1 <= s.length, t.length <= 200
- s and t only contain lowercase letters and '#' characters.
- Follow up: Can you solve it in O(n) time and O(1) space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

'#' is a backspace character that deletes the previous character (if any).

We simulate typing both strings into an empty text editor and return

true if the final texts are equal. Right?"

"A few quick questions:

What if '#' appears at the start with nothing to delete? It's a no-op.

Can strings be empty? Yes — empty result equals empty result.

The follow-up asks for O(n) time and O(1) space."

"Let me walk: s='ab#c', t='ad#c'.

s: a,b,# -> a, c -> 'ac'. t: a,d,# -> a, c -> 'ac'. Equal -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build s and t as strings with a stack (push letters,
    pop on '#')
return the two results are equal
```

TIME	SPACE
O(n + m)	O(n + m)

WHY IT IS SLOW

Linear, but stores both processed strings. Walk both strings from the end with two pointers, skipping characters cancelled by '#': O(n + m) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n+m) extra space for the stacks.

Scan both strings from the RIGHT with two pointers.

Use a skip counter: every '#' increments skip; a regular char while skip>0 is skipped.

The first non-skipped char from each string is compared.

We don't need any extra storage — just integer counters.

Time: O(n+m). Space: O(1).

Does that make sense before I code it?"

Key insight: scanning right-to-left with a skip counter avoids building processed strings — a '#' increments skip; a letter with skip > 0 is consumed; a letter with skip = 0 is the next character to compare.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _844_BackspaceStringCompare:
    def backspaceCompare(self, s, t):
        i, j = len(s) - 1, len(t) - 1
        skip_s = skip_t = 0
        while i >= 0 or j >= 0:
            while i >= 0:
                if s[i] == '#': skip_s += 1; i -= 1
                elif skip_s > 0: skip_s -= 1; i -= 1
                else: break
            while j >= 0:
                if t[j] == '#': skip_t += 1; j -= 1
                elif skip_t > 0: skip_t -= 1; j -= 1
                else: break
            if i >= 0 and j >= 0 and s[i] != t[j]:
                return False
            if (i >= 0) != (j >= 0):
                return False
            i -= 1; j -= 1
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='ab#c', t='ad#c' (i=3,j=3):

s[3]='c' non-# -> compare. t[3]='c' -> compare. 'c'=='c' ok. i=2,j=2.

s[2]='#' -> skip_s=1, i=1. s[1]='b', skip_s>0 -> skip, i=0. s[0]='a' non-# -> compare.

t[2]='#' -> skip_t=1, j=1. t[1]='d', skip -> j=0. t[0]='a' -> compare. 'a'=='a' ok. i=-1,j=-1.

both exhausted -> True ok

The (i>=0) != (j>=0) check catches mismatched remaining lengths

PHASE 6 COMPLEXITY & FOLLOW-UP

Stack approach: O(n+m) time, O(n+m) space — simple and readable.

Two-pointer approach: O(n+m) time, O(1) space — optimal.

Follow-up: if '#' can delete k characters at once, adjust skip counter to add k.

Follow-up: Minimum Remove to Make Valid Parentheses (#1249) — similar stack-simulation idea.

EASY | Stack | LeetCode #933 | 10 of 13 | 10/13

Number of Recent Calls

QUESTION

You have a RecentCounter class which counts the number of recent requests within a certain time frame.

Implement the RecentCounter class:

CONSTRAINTS

- RecentCounter() Initializes the counter with zero recent requests.
- int ping(int t) Adds a new request at time t, where t represents some time in milliseconds, and returns the number of requests that has happened in the past 3000 milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range [t - 3000, t].

It is guaranteed that every call to ping uses a strictly larger value of t than the previous call.

- $1 <= t <= 10^9$
- Each test case will call ping with strictly increasing values of t.
- At most 10^4 calls will be made to ping.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement a recent call counter: ping(t) records a call at time t and returns how many calls

are in the last 3000ms (inclusive). Right?"

"A few quick questions:

Pings always arrive in increasing order? Yes. Window is [t-3000, t]? Yes."

"Let me walk: ping(1)=1, ping(100)=2, ping(3001)=3, ping(3002)=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def ping(t):
    append t to a list
    count the entries in [t - 3000, t] by
        scanning the whole list
```

TIME	SPACE
O(n) per call	O(n)

WHY IT IS SLOW

Rescans all calls, and the list grows forever. Keep a queue and pop the front while it is older than t - 3000: the answer is the queue length, O(1) amortised.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Queue with timestamp expiry — O(1) amortized. Walk-through:"

self.q = deque() storing all ping timestamps.

ping(t):

q.append(t).

Remove all timestamps older than t-3000 from the front:

while q and q[0] <= t - 3000: q.popleft().

Return len(q).

WHY deque: append to right (new pings), remove from left (expired pings) -> both O(1).

WHY <= t-3000: pings within [t-2999, t] are valid (last 3000 seconds, inclusive of both ends).

Amortized O(1): each timestamp is added once and removed once.

Time: O(1) amortized. Space: O(size of window).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _933_NumberOfRecentCalls:
    def __init__(self):
        self._queue = deque()

    def ping(self, t):
        self._queue.append(t)
        while self._queue[0] < t - 3000:
            self._queue.popleft()
        return len(self._queue)
```

PHASE 5 TEST & DEBUG

"Let me trace RecentCounter: ping(1),ping(100),ping(3001),ping(3002)."

Maintain deque of timestamps; keep only those in [t-3000, t]

ping(1): deque=[1]. Count=1 ok

ping(100): deque=[1,100]. Count=2 ok

ping(3001): deque=[1,100,3001]. Remove < 3001-3000=1: remove 1? 1==1 so keep? Actually <1 none removed. deque=[1,100,3001]. Count=3 ok

ping(3002): deque=[1,100,3001,3002]. Remove <2: remove 1. deque=[100,3001,3002]. Count=3 ok

WHY deque: O(1) front removal; sliding window maintained automatically.

Timestamps guaranteed increasing; only front-pop needed (no out-of-order elements).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) amortized: each ping added/removed at most once	O(w) window size

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: sliding window median — requires two heaps or sorted structure.

Follow-up: Hit Counter (#362) — similar, but queries ask about arbitrary past windows.

IN YOUR APP | Expose | Editorial | Community

EASY | Stack | LeetCode #1047 | 11 of 13 | 11/13

Remove All Adjacent Duplicates In String

QUESTION

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on s until we no longer can.

Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1

Input	`s = "abbaca"`
Output	`"ca"`
Explanation	*For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".*

Example 2

Input	`s = "azxxzy"`
Output	`"ay"`

CONSTRAINTS

- 1 <= s.length <= 10^5
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Repeatedly remove all adjacent duplicate pairs until no more exist. Return result. Right?"

"A few quick questions:

Process left to right; removing a pair may create new adjacent duplicates? Yes.

Return the final string after all possible removals."

"Let me walk: s='abbaca' -> remove 'bb'->'aaca' -> remove 'aa'->'ca' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    scan the string and delete the first pair of
        equal adjacent letters
    if no pair remains:
        break
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each deletion rebuilds the string and rescans from the start. A stack: push a letter unless it equals the top (then pop): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is repeated scanning.

A stack processes this in one pass:

Push each character. If the stack top equals current char, pop (remove pair).

Otherwise push. Result is the final stack joined.

Time: O(n). Space: O(n)."

Key insight: the stack maintains a "collapsed" version of the string — a character matching the top is its duplicate and both are eliminated immediately, giving O(n) without backtracking.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1047_RemoveAdjacentDuplicates:
    def removeDuplicates(self, s):
        stack = []
        for ch in s:
            if stack and stack[-1] == ch:
                stack.pop()
            else:
                stack.append(ch)
        return ''.join(stack)
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='abbaca': a->[a]. b->[a,b]. b:top='b'==b->pop->[a]. a:top='a'==a->pop->[]. c->[c]. a->[c,a]. -> 'ca' ok

s='azxxzy': a,z->[a,z]. x->[a,z,x]. x:pop->[a,z]. z:pop->[a]. y->[a,y]. -> 'ay' ok

Each push/pop pair handles one duplicate removal — chain removals are automatic

Edge: s='aaa' -> 'a','a' pop -> '', 'a' push -> 'a' ok (odd duplicates leave one). s='' -> '' ok. Each char processed exactly once -> O(n) time.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each char pushed and popped at most once	**O(n)**

Follow-up: Remove All Adjacent Duplicates II (#1209) — remove groups of k identical chars.

Stack stores (char, count); when count reaches k, pop the group.

IN YOUR APP | Expose | Editorial | Community

EASY | Stack | LeetCode #1475 | 12 of 13 | 12/13

Final Prices With a Special Discount in a Shop

QUESTION

You are given an integer array prices where prices[i] is the price of the ith item in a shop.

There is a special discount for items in the shop. If you buy the ith item, then you will receive a discount equivalent to prices[j] where j is the minimum index such that j > i and prices[j] <= prices[i]. Otherwise, you will not receive any discount at all.

Return an integer array answer where answer[i] is the final price you will pay for the ith item of the shop, considering the special discount.

Example 1

Input	`prices = [8,4,6,2,3]`
Output	`[4,2,4,2,3]`
Explanation	*For item 0 with price[0]=8 you will receive a discount equivalent to prices[1]=4, therefore, the final price you will pay is 8 - 4 = 4.* *For item 1 with price[1]=4 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 4 - 2 = 2.* *For item 2 with price[2]=6 you will receive a discount equivalent to prices[3]=2, therefore, the final price you will pay is 6 - 2 = 4.* *For items 3 and 4 you will not receive any discount at all.*

Example 2

Input	`prices = [1,2,3,4,5]`
Output	`[1,2,3,4,5]`
Explanation	*In this case, for all items, you will not receive any discount at all.*

Example 3

Input	`prices = [10,1,1,6]`
Output	`[9,0,1,6]`

CONSTRAINTS

- 1 <= prices.length <= 500
- 1 <= prices[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each item, find the final price = price - max discount where discount is the first item

at the same or lower price appearing later in the array. Right?"

"A few quick questions:

The discount is the FIRST subsequent item with price <= prices[i]? Yes."

"Let me walk: prices=[8,4,6,2,3] -> 8->8-4=4, 4->4-2=2, 6->6-2=4, 2->2-2=0, 3->3. -> [4,2,4,0,3] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for item in items:
    scan the later items for the first price that
        is less than or equal to it
    subtract that discount
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per item. A monotonic stack of indices: when a price arrives, pop every item whose price is at least as large and apply the discount: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Monotonic stack (non-increasing) scanning right-to-left — O(n). Walk-through:"

Process prices from right to left. Stack stores prices.

For each price i from right to left:

While stack non-empty and stack[-1] > prices[i]: stack.pop().

`result[i] = prices[i] - stack[-1] if stack else prices[i].`

<- stack[-1] = nearest future price <= prices[i] = the special discount.

`stack.push(prices[i]).`

WHY non-increasing stack: we want the FIRST (nearest) future price <= current.

Larger future prices are useless (they don't give a discount) -> pop them.

WHY right-to-left: discount comes from future prices (to the right).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1475_FinalPricesWithSpecialDiscount:
    def finalPrices(self, prices):
        result = list(prices)
        stack = []
        for i, price in enumerate(prices):
            while stack and prices[stack[-1]] >= price:
                j = stack.pop()
                result[j] = prices[j] - price
            stack.append(i)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

prices=[8,4,6,2,3]:

i=0(8): stack=[0]. i=1(4): 8>=4->pop0,result[0]=8-4=4. stack=[1].

i=2(6): 4<6->push. stack=[1,2]. i=3(2): 6>=2->pop2,result[2]=6-2=4. 4>=2->pop1,result[1]=4-2=2.

stack=[3]. i=4(3): 2<3->push. stack=[3,4]. Remaining: result[3]=2,result[4]=3. -> [4,2,4,2,3]?

Wait: should be [4,2,4,0,3]. Let me retrace: i=3(2): pop2->6-2=4 ok. pop1: prices[1]=4>=2->4-2=2 ok. stack=[3].

i=4(3): prices[3]=2<3->push. stack=[3,4]. result[3]=2(original,no discount yet).

Hmm: price[3]=2 and price[4]=3. 2 <= 3? No wait: condition is prices[stack[-1]] >= price.

prices[3]=2, price=3. 2>=3? No -> don't pop. result[3] stays 2. -> [4,2,4,2,3]?

But expected [4,2,4,0,3]. Oh wait: prices=[8,4,6,2,3], price[3]=2 is discounted by price[3] itself? No.

Actually prices[3]=2, check forward: price[4]=3>2 so no discount. result[3]=2-0=2. Expected says 0?

Let me recheck: 8-4=4 ok, 4-2=2 ok, 6-2=4 ok, 2-2=0?? price[3]=2, discount is FIRST later <=2. price[4]=3>2 so no discount -> result[3]=2. Not 0. Example output must be different. -> [4,2,4,2,3] ok

Stack correctly finds first future price <= current

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each index pushed/popped once	**O(n)**

This is 'next smaller or equal element' — same family as Daily Temperatures.

Follow-up: Next Greater Element I (#496) — next strictly greater.

Follow-up: Largest Rectangle in Histogram (#84) — monotonic stack for spans.

EASY | Stack | LeetCode #1614 | 13 of 13 | 13/13

Maximum Nesting Depth of the Parentheses

QUESTION

Given a valid parentheses string s, return the nesting depth of s. The nesting depth is the maximum number of nested parentheses.

Example 1	
Input	`s = "(1+(2*3)+((8)/4))+1"`
Output	3
Explanation	*Digit 8 is inside of 3 nested parentheses in the string.*

Example 2	
Input	`s = "(1)+((2))+(((3)))"`
Output	3
Explanation	*Digit 3 is inside of 3 nested parentheses in the string.*

Example 3	
Input	`s = "()(())((()()))"`
Output	3

CONSTRAINTS

- 1 <= s.length <= 100
- s consists of digits 0-9 and characters '+', '-', '*', '/', '(', and ')'.
- It is guaranteed that parentheses expression s is a VPS.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the maximum nesting depth of parentheses in the given valid expression. Right?"

"A few quick questions:

Only '(' and ')' contribute to depth; digits and operators are ignored? Yes.

Input is always a valid expression? Yes."

"Let me walk: s='(1+(2*3)+((8)/4))+1' -> max depth is 3 (the '((8)/4)' part) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
for i in range(n):
    count the unclosed '(' before it
    the answer is the largest count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Recounts the prefix for every position. Keep one counter, +1 on '(' and -1 on ')', and track its maximum: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single scan tracking current depth — O(n). Walk-through:"

depth=0, max_depth=0.

For each char:

'(': depth++; max_depth = max(max_depth, depth).

')': depth--.

Return max_depth.

HOW depth tracks nesting: each '(' opens a new level; each ')' closes.

`max_depth = deepest we ever went.`

WHY depth always >= 0: problem guarantees valid parentheses string.

No stack needed — just a counter, since we only need the maximum, not the actual structure.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1614_MaxNestingDepth:
    def maxDepth(self, s):
        current_depth = 0
        max_depth = 0
        for ch in s:
            if ch == '(':
                current_depth += 1
                max_depth = max(max_depth, current_depth)
            elif ch == ')':
                current_depth -= 1
        return max_depth
```

PHASE 5 TEST & DEBUG

"Let me trace the stack actions (push/pop) and how they reflect the structure."

s='(1+(2*3)+((8)/4))+1':

'('->depth=1,max=1. '('->2,max=2. ')'->1. '('->2. '('->3,max=3. ')'->2. ')'->1. ')'->0. -> 3 ok

PHASE 5 TEST & DEBUG

s='()(())': '('->1,max=1. ')'->0. '('->1. '('->2,max=2. ')'->1. ')'->0. -> 2 ok

Track depth with a counter; max is updated only on '('

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: Minimum Add to Make Parentheses Valid (#921) — count unmatched brackets.

Follow-up: split balanced parentheses into groups by depth level.

IN YOUR APP | Expose | Editorial | Community